CKA

Certified Kubernetes Administrator

Complete Preparation Guide · Questions · Commands · Tips · Tricks

EXAM FORMAT	PASS MARK	DOMAINS
2 Hours · 15–20 Tasks Performance-based CLI	66% or above One free retake	5 Weighted Domains kubectl · YAML · etcd

Domain	Weight	Key Topics
Cluster Architecture, Installation & Configuration	25%	kubeadm, etcd backup/restore, RBAC, certificates, HA clusters
Workloads & Scheduling	15%	Deployments, DaemonSets, Jobs, resource limits, taints/tolerations, affinity
Services & Networking	20%	ClusterIP, NodePort, LoadBalancer, Ingress, NetworkPolicy, DNS, CoreDNS
Storage	10%	PV, PVC, StorageClass, volume types, access modes
Troubleshooting	30%	Node issues, pod crashes, networking failures, logs, events, metrics-server

★ EXAM CCFBF1

CHAPTER 1 · EXAM STRATEGY & ENVIRONMENT SETUP

Time management · aliases · imperative vs declarative · kubectl config tips

1.1 The First 5 Minutes — Environment Setup

Before touching a single question, spend 2–3 minutes setting up your shell. This pays dividends throughout the entire 2-hour exam.

```
# 1. Set the most important alias of your life
alias k=kubectl
export do='--dry-run=client -o yaml'
export now='--force --grace-period 0'

# 2. Enable kubectl autocomplete
source <(kubectl completion bash)
echo 'source <(kubectl completion bash)' >> ~/.bashrc
complete -o default -F __start_kubectl k

# 3. Set your preferred editor
export KUBE_EDITOR='nano'    # or vim

# 4. Verify cluster context before EVERY question
kubectl config get-contexts
kubectl config use-context <context-name>
kubectl config current-context
```

⚠ WATCH OUT FEE2E2

1.2 Imperative vs Declarative — Know When to Use Each

The exam rewards speed. Typing YAML from scratch wastes time. Use imperative commands to generate YAML, then edit.

Situation	Approach	Speed
Simple pod / deployment creation	Imperative: kubectl run / kubectl create	Fastest
Need to customise YAML fields	Imperative --dry-run=client -o yaml > file.yaml, then edit	Fast
Complex multi-container / volumes	Write YAML from scratch or use docs template	Slower but accurate
Modify existing resource	kubectl edit <resource> or kubectl patch	Fast
Apply idempotently	kubectl apply -f file.yaml	Standard

1.3 Time Management Strategy

Phase	Action	Time Budget
Read question	Identify: cluster context, namespace, resource type, specific requirement	30 sec
Switch context	kubectl config use-context <ctx>	10 sec
Solve	Run commands, write YAML, verify	3–8 min per Q
Verify	kubectl get / describe / logs to confirm solution works	1–2 min

Flag & skip	If stuck after 4 min, flag and move on — return at end	Save 10+ min

★ EXAM CCFBF1

1.4 Essential kubectl Shortcuts Reference

```
# Short resource names (use these — saves typing)
po    = pods              svc   = services          ns   = namespaces
deploy = deployments      ds    = daemonsets         rs   = replicasets
cm    = configmaps        sa    = serviceaccounts    pv   = persistentvolumes
pvc   = persistentvolumeclaims    ep = endpoints     ing = ingresses
netpol = networkpolicies  csr   = certificatesigningrequests

# Common output formats
kubectl get pods -o wide           # show node and IP
kubectl get pods -o yaml           # full YAML
kubectl get pods -o json           # JSON output
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get events --sort-by=.lastTimestamp

# Quick resource info without docs
kubectl explain pod.spec.containers
kubectl explain deployment.spec.strategy
kubectl api-resources    # list all resource types + short names
```

CHAPTER 2 · WORKLOADS & SCHEDULING — 15%

Pods · Deployments · DaemonSets · Jobs · Resource Limits · Taints · Affinity · Labels

Q1 Create a pod named 'nginx-pod' using image nginx:1.21 in namespace 'web', with label app=frontend, and resource limits of 200m CPU and 128Mi memory.

▸ **ANSWER**

Use the imperative command with --dry-run to generate the YAML, then patch in resource limits. This is faster than writing from scratch.

⌨ COMMAND / YAML SNIPPET

```
# Step 1: Generate base YAML
kubectl run nginx-pod --image=nginx:1.21 --namespace=web \
  --labels=app=frontend --dry-run=client -o yaml > pod.yaml

# Step 2: Add resources section — edit pod.yaml
# Under containers[0] add:
    resources:
      limits:
        cpu: '200m'
```

```
          memory: '128Mi'
        requests:
          cpu: '100m'
          memory: '64Mi'

# Step 3: Apply and verify
kubectl apply -f pod.yaml
kubectl get pod nginx-pod -n web
kubectl describe pod nginx-pod -n web | grep -A5 Limits
```

★ **EXAM TIP** If the namespace does not exist: kubectl create namespace web first. Always verify with kubectl describe.

Q2 Create a Deployment named 'webapp' with 3 replicas of image httpd:2.4 and then scale it to 5 replicas. Configure a rolling update strategy with maxSurge=1 and maxUnavailable=0.

▸ **ANSWER**

Generate deployment imperatively, then edit strategy. Rolling update with maxUnavailable=0 ensures zero-downtime deployments.

⌨ COMMAND / YAML SNIPPET

```
# Create deployment
kubectl create deployment webapp --image=httpd:2.4 --replicas=3

# Scale to 5 (imperative — fastest)
kubectl scale deployment webapp --replicas=5

# Edit rolling update strategy
kubectl edit deployment webapp
# Find spec.strategy and set:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0

# Verify
kubectl get deployment webapp
kubectl describe deployment webapp | grep -A4 Strategy
kubectl rollout status deployment/webapp
```

□ **PRO TIP** kubectl rollout history deployment/webapp shows revision history. kubectl rollout undo deployment/webapp rolls back.

Q3 A deployment 'api-server' is in CrashLoopBackOff. Troubleshoot and fix it — the container is failing because the wrong command is set.

▸ **ANSWER**

CrashLoopBackOff means the container starts, crashes, and Kubernetes keeps restarting it. Debug sequence: describe → logs → edit fix.

⌨ COMMAND / YAML SNIPPET

```
# Step 1: Check pod status and events
kubectl describe pod <pod-name> | grep -A20 'Events'
kubectl describe pod <pod-name> | grep -A5 'Command'

# Step 2: Check container logs
kubectl logs <pod-name>                    # current logs
kubectl logs <pod-name> --previous         # logs from crashed container

# Step 3: Fix — edit deployment
kubectl edit deployment api-server
# Remove or correct the spec.containers[0].command field

# Step 4: Verify recovery
kubectl get pods -w    # watch pods recover
kubectl rollout status deployment/api-server
```

⚠ WATCH OUT Always check BOTH current logs AND --previous logs. The crash message is in the previous container's logs, not the current one.

Q4 Create a DaemonSet that runs a log-collector pod (image: fluentd:v1.14) on every node EXCEPT the control-plane node.

▸ **ANSWER**

DaemonSets run one pod per node. To exclude the control-plane, use a toleration for the control-plane taint. Actually, the default is to NOT schedule on control-plane — you need a toleration TO include it. So just create a basic DaemonSet.

⌨ COMMAND / YAML SNIPPET

```
# DaemonSets have no imperative create command — use YAML
# Generate base from docs or use this template:
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
spec:
  selector:
    matchLabels:
      app: log-collector
  template:
    metadata:
      labels:
        app: log-collector
    spec:
      tolerations: []   # empty = no control-plane scheduling
      containers:
      - name: fluentd
        image: fluentd:v1.14
EOF

kubectl get daemonset log-collector
kubectl get pods -o wide | grep log-collector  # verify one per node
```

★ **EXAM TIP** To INCLUDE control-plane nodes add: tolerations: [{key: node-role.kubernetes.io/control-plane, operator: Exists, effect: NoSchedule}]

Q5 Create a Job named 'batch-pi' that calculates pi using image perl:5.34, runs the command perl -Mbignum=bpi -wle 'print bpi(2000)', with completions=3 and parallelism=2.

▸ **ANSWER**

Jobs run a task to completion. completions=how many times total, parallelism=how many run at once.

COMMAND / YAML SNIPPET

```
cat <<EOF | kubectl apply -f -
apiVersion: batch/v1
kind: Job
metadata:
  name: batch-pi
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.34
        command: ['perl', '-Mbignum=bpi', '-wle', 'print bpi(2000)']
EOF

kubectl get jobs batch-pi -w
kubectl get pods -l job-name=batch-pi
kubectl logs <completed-pod-name>
```

⚠ **WATCH OUT** restartPolicy for Jobs must be Never or OnFailure — never Always. CronJobs use the same template but add schedule: '*/1 * * * *'

Q6 Schedule a pod to run ONLY on nodes with label disktype=ssd. Then schedule another pod to PREFER nodes with region=us-east but fall back to others.

▸ **ANSWER**

nodeSelector for hard requirement. nodeAffinity requiredDuringScheduling for hard, preferredDuringScheduling for soft.

COMMAND / YAML SNIPPET

```
# Hard requirement — nodeSelector (simplest)
kubectl run ssd-pod --image=nginx --dry-run=client -o yaml > ssd.yaml
# Add to spec:
  nodeSelector:
    disktype: ssd

# Soft preference — nodeAffinity
  affinity:
```

```
      nodeAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions:
            - key: region
              operator: In
              values: [us-east]

  # Label a node to test
  kubectl label node <node-name> disktype=ssd
  kubectl get nodes --show-labels | grep disktype
```

★ **EXAM TIP** requiredDuringSchedulingIgnoredDuringExecution = hard (pod stays unscheduled if no match). preferredDuring = soft (best effort).

Q7 Taint a node 'worker-1' with key=value:NoSchedule. Then create a pod that tolerates this taint and verify it lands on that node.

▸ **ANSWER**

Taints repel pods. Tolerations allow pods to be scheduled on tainted nodes. Effect: NoSchedule, PreferNoSchedule, NoExecute.

⌨ **COMMAND / YAML SNIPPET**

```
  # Taint the node
  kubectl taint node worker-1 env=prod:NoSchedule

  # Verify taint
  kubectl describe node worker-1 | grep Taint

  # Create pod with toleration
  kubectl run tolerant-pod --image=nginx --dry-run=client -o yaml > t.yaml
  # Add to spec:
    tolerations:
    - key: 'env'
      operator: 'Equal'
      value: 'prod'
      effect: 'NoSchedule'
    nodeName: worker-1   # optional: force to that node

  kubectl apply -f t.yaml
  kubectl get pod tolerant-pod -o wide   # verify NODE column

  # Remove taint later (note the trailing minus)
  kubectl taint node worker-1 env=prod:NoSchedule-
```

 PRO TIP NoExecute evicts EXISTING pods if they don't tolerate. NoSchedule only prevents NEW scheduling. PreferNoSchedule is best-effort.

CHAPTER 3 · SERVICES & NETWORKING — 20%

Services · Ingress · NetworkPolicy · DNS · CoreDNS · Endpoints

Q8 Expose a deployment 'myapp' running on port 8080 via a ClusterIP service on port 80. Then expose it externally via NodePort on port 30080.

▸ **ANSWER**

kubectl expose is the fastest way. ClusterIP is default and internal-only. NodePort exposes on every node's IP at the specified port.

⌨ COMMAND / YAML SNIPPET

```
# ClusterIP service (internal only)
kubectl expose deployment myapp --port=80 --target-port=8080 --name=myapp-
svc

# NodePort service
kubectl expose deployment myapp --port=80 --target-port=8080 \
  --type=NodePort --name=myapp-nodeport

# To set a SPECIFIC NodePort (30000-32767 range), use YAML:
kubectl expose deployment myapp --port=80 --target-port=8080 \
  --type=NodePort --dry-run=client -o yaml > svc.yaml
# Add under spec.ports[0]: nodePort: 30080
kubectl apply -f svc.yaml

# Verify
kubectl get svc myapp-nodeport
kubectl describe svc myapp-nodeport
curl <node-ip>:30080
```

★ **EXAM TIP** Service types: ClusterIP (default, internal), NodePort (external via node), LoadBalancer (cloud LB), ExternalName (DNS alias).

Q9 Create an Ingress resource that routes traffic: /api → backend-svc:8080, /web → frontend-svc:80. Use host: myapp.example.com.

▸ **ANSWER**

Ingress requires an Ingress Controller to be running (nginx-ingress). The Ingress resource just defines routing rules.

⌨ COMMAND / YAML SNIPPET

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
```

```
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: backend-svc
            port:
              number: 8080
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
EOF

kubectl get ingress myapp-ingress
kubectl describe ingress myapp-ingress
```

 PRO TIP pathType: Prefix matches /api and /api/v1/anything. pathType: Exact matches ONLY /api exactly. Always check which the question requires.

Q10 Create a NetworkPolicy that allows pods in the 'frontend' namespace to reach pods in 'backend' namespace on port 5432 only. Deny all other ingress to backend.

▸ **ANSWER**

NetworkPolicy requires a CNI plugin that supports it (Calico, Cilium, Weave). Default deny then explicit allow is the pattern.

COMMAND / YAML SNIPPET

```
# Apply to the 'backend' namespace — controls what enters backend pods
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: backend
spec:
  podSelector: {}       # applies to ALL pods in backend namespace
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: frontend
    ports:
    - protocol: TCP
      port: 5432
```

```
EOF

# Label the frontend namespace if not already labelled
kubectl label namespace frontend kubernetes.io/metadata.name=frontend
kubectl get networkpolicy -n backend
```

⚠ **WATCH OUT** Gotcha: from: [{namespaceSelector:...}, {podSelector:...}] is OR logic. from: [{namespaceSelector:..., podSelector:...}] (same list item) is AND logic.

Q11 Troubleshoot: a pod cannot resolve the service 'database-svc.prod.svc.cluster.local'. Diagnose and fix the DNS issue.

▸ **ANSWER**

Kubernetes DNS follows pattern: <service>.<namespace>.svc.<cluster-domain>. CoreDNS runs in kube-system and handles all in-cluster DNS.

⌨ **COMMAND / YAML SNIPPET**

```
# Step 1: Test DNS from inside a temporary pod
kubectl run dns-test --image=busybox:1.28 --rm -it --restart=Never \
  -- nslookup database-svc.prod.svc.cluster.local

# Step 2: Check CoreDNS pods are running
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns

# Step 3: Check CoreDNS ConfigMap
kubectl get configmap coredns -n kube-system -o yaml

# Step 4: Check if service exists and has endpoints
kubectl get svc database-svc -n prod
kubectl get endpoints database-svc -n prod

# Step 5: Check pod DNS config
kubectl exec <pod-name> -- cat /etc/resolv.conf

# DNS shortnames that work from within same namespace:
# database-svc  (same namespace)
# database-svc.prod  (cross-namespace short)
# database-svc.prod.svc.cluster.local  (full FQDN)
```

⚠ **WATCH OUT** Empty Endpoints means no pods match the service selector. Check: kubectl describe svc database-svc and compare selector to pod labels.

CHAPTER 4 · STORAGE — 10%

PersistentVolumes · PVCs · StorageClass · Volume Mounts · emptyDir · hostPath · ConfigMap volumes

Q12 Create a PersistentVolume of 1Gi using hostPath /mnt/data with ReadWriteOnce access mode. Then create a PVC that claims 500Mi from it.

▸ **ANSWER**

PV is a cluster-wide resource (no namespace). PVC is namespaced and claims storage. A PVC binds to a PV when capacity and access mode match.

COMMAND / YAML SNIPPET

```
# Create PersistentVolume
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  hostPath:
    path: /mnt/data
  persistentVolumeReclaimPolicy: Retain
EOF

# Create PersistentVolumeClaim
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
  namespace: default
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
EOF

kubectl get pv my-pv          # should show Bound
kubectl get pvc my-pvc        # should show Bound
```

★ **EXAM TIP** Access modes: ReadWriteOnce (RWO) = one node. ReadOnlyMany (ROX) = many nodes read. ReadWriteMany (RWX) = many nodes write. Reclaim: Retain, Delete, Recycle.

Q13 Mount a PVC into a pod at /data and also mount a ConfigMap as a volume at /config. The ConfigMap contains a file app.properties.

▸ **ANSWER**

Pods can mount multiple volume types simultaneously. ConfigMaps become files in the mount directory — each key is a filename.

COMMAND / YAML SNIPPET

```
# Create ConfigMap first
kubectl create configmap app-config \
  --from-literal=app.properties='db.host=localhost\ndb.port=5432'

# Create pod with both mounts
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: multi-mount-pod
spec:
  volumes:
  - name: data-vol
    persistentVolumeClaim:
      claimName: my-pvc
  - name: config-vol
    configMap:
      name: app-config
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - name: data-vol
      mountPath: /data
    - name: config-vol
      mountPath: /config
EOF

kubectl exec multi-mount-pod -- ls /config
kubectl exec multi-mount-pod -- cat /config/app.properties
```

PRO TIP Secrets work exactly the same as ConfigMaps for volume mounts — just replace configMap: with secret: and name: <secret-name>.

Q14 Create a StorageClass named 'fast-storage' using the kubernetes.io/no-provisioner provisioner with WaitForFirstConsumer binding mode. Then create a PVC using this StorageClass.

▸ **ANSWER**

StorageClass defines how storage is provisioned. WaitForFirstConsumer delays binding until a pod using the PVC is scheduled — important for topology awareness.

COMMAND / YAML SNIPPET

```
cat <<EOF | kubectl apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
EOF
```

```
# Create PVC referencing the StorageClass
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: fast-pvc
spec:
  storageClassName: fast-storage
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
EOF

kubectl get storageclass
kubectl get pvc fast-pvc   # will show Pending until pod schedules
```

★ **EXAM TIP** Immediate binding (default) binds PVC to PV immediately. WaitForFirstConsumer waits until a pod needs it — avoids zone mismatch in multi-AZ clusters.

CHAPTER 5 · CLUSTER ARCHITECTURE, INSTALLATION & CONFIG — 25%

kubeadm · etcd · RBAC · Certificates · Cluster Upgrade · HA

Q15 Back up the etcd cluster and restore it from a backup. The etcd is running as a static pod. Provide the exact commands.

▸ **ANSWER**

etcd is the brain of Kubernetes — backing it up is a critical CKA topic. You WILL get an etcd backup/restore question. Know this perfectly.

⌨ COMMAND / YAML SNIPPET

```
# Step 1: Find etcd endpoints and certificates
kubectl describe pod etcd-controlplane -n kube-system | grep -E 'listen-client|cert-file|key-file|trusted-ca'
# OR check the static pod manifest:
cat /etc/kubernetes/manifests/etcd.yaml | grep -E 'endpoints|cert|key|ca'

# Step 2: Take backup using etcdctl
ETCDCTL_API=3 etcdctl snapshot save /opt/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Step 3: Verify backup
ETCDCTL_API=3 etcdctl snapshot status /opt/etcd-backup.db --write-out=table
```

```
# Step 4: Restore from backup
ETCDCTL_API=3 etcdctl snapshot restore /opt/etcd-backup.db \
  --data-dir=/var/lib/etcd-restore

# Step 5: Update etcd static pod to use new data dir
vi /etc/kubernetes/manifests/etcd.yaml
# Change: --data-dir=/var/lib/etcd  TO  --data-dir=/var/lib/etcd-restore
# Also update hostPath volume to point to /var/lib/etcd-restore

# Step 6: Wait for etcd to restart (watch the static pod)
watch kubectl get pods -n kube-system | grep etcd
```

⚠ **WATCH OUT** Always export ETCDCTL_API=3 before any etcdctl command. etcdctl v2 API is deprecated. The certs path is almost always /etc/kubernetes/pki/etcd/.

Q16 Create a ClusterRole that allows get, list, watch on pods and deployments. Bind it to a ServiceAccount 'ci-bot' in namespace 'devops'.

▸ **ANSWER**

RBAC is always on the CKA. ClusterRole = cluster-wide permissions. ClusterRoleBinding = binds it cluster-wide. RoleBinding with ClusterRole = namespaced binding.

⌨ **COMMAND / YAML SNIPPET**

```
# Create ClusterRole
kubectl create clusterrole pod-reader \
  --verb=get,list,watch \
  --resource=pods,deployments

# Create ServiceAccount
kubectl create serviceaccount ci-bot -n devops

# Bind ClusterRole to ServiceAccount (RoleBinding = namespace-scoped)
kubectl create rolebinding ci-bot-binding \
  --clusterrole=pod-reader \
  --serviceaccount=devops:ci-bot \
  --namespace=devops

# Verify permissions
kubectl auth can-i get pods \
  --as=system:serviceaccount:devops:ci-bot -n devops
kubectl auth can-i delete pods \
  --as=system:serviceaccount:devops:ci-bot -n devops   # should be no

# View all bindings for the SA
kubectl get rolebindings -n devops -o yaml | grep -A5 ci-bot
```

★ **EXAM TIP** Use RoleBinding (namespaced) when you want to restrict to one namespace even with a ClusterRole. Use ClusterRoleBinding for cluster-wide access.

Q17 Upgrade a kubeadm cluster from v1.29.x to v1.30.x — upgrade the control plane first, then the worker node.

▸ **ANSWER**

Cluster upgrade is a guaranteed CKA question. The sequence is always: upgrade kubeadm → plan → apply control plane → drain node → upgrade kubelet/kubectl → uncordon.

COMMAND / YAML SNIPPET

```
# ═══ CONTROL PLANE NODE ═══
# Step 1: Check current version
kubectl get nodes
kubeadm version

# Step 2: Upgrade kubeadm package
apt-mark unhold kubeadm
apt-get update && apt-get install -y kubeadm=1.30.0-1.1
apt-mark hold kubeadm
kubeadm version   # verify

# Step 3: Plan and apply upgrade
kubeadm upgrade plan
kubeadm upgrade apply v1.30.0

# Step 4: Upgrade kubelet and kubectl on control plane
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.30.0-1.1 kubectl=1.30.0-1.1
apt-mark hold kubelet kubectl
systemctl daemon-reload && systemctl restart kubelet

# ═══ WORKER NODE (run from control plane) ═══
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# ═══ ON WORKER NODE (SSH in) ═══
apt-mark unhold kubeadm && apt-get install -y kubeadm=1.30.0-1.1
kubeadm upgrade node
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.30.0-1.1 kubectl=1.30.0-1.1
systemctl daemon-reload && systemctl restart kubelet

# ═══ BACK ON CONTROL PLANE ═══
kubectl uncordon worker-1
kubectl get nodes   # verify all nodes show v1.30.x
```

⚠ **WATCH OUT** CRITICAL: drain before upgrade, uncordon after. Never skip drain — it evicts pods safely. Upgrade one minor version at a time — cannot skip versions.

Q18 A node 'worker-2' is NotReady. Diagnose and recover it.

▸ **ANSWER**

NotReady means the kubelet on that node is not communicating with the API server. SSH to the node and check kubelet service.

COMMAND / YAML SNIPPET

```
# Step 1: From control plane — check node
kubectl get nodes
kubectl describe node worker-2 | grep -A20 Conditions
kubectl get events --field-selector involvedObject.name=worker-2

# Step 2: SSH to the worker node
ssh worker-2

# Step 3: Check kubelet service
systemctl status kubelet
journalctl -u kubelet -n 50 --no-pager   # last 50 log lines

# Step 4: Common fixes
# If kubelet is stopped:
systemctl start kubelet
systemctl enable kubelet

# If config issue — check kubelet config
cat /etc/kubernetes/kubelet.conf
cat /var/lib/kubelet/config.yaml

# If container runtime issue:
systemctl status containerd
systemctl restart containerd

# Step 5: Verify from control plane
kubectl get nodes   # worker-2 should become Ready
```

PRO TIP Most NotReady causes: kubelet stopped, wrong --node-ip, certificate expired, containerd/docker runtime down, disk pressure, memory pressure.

Q19 Create a certificate signing request (CSR) for a user 'john' and approve it. Then create a kubeconfig for john with limited access.

▸ **ANSWER**

CKA tests your understanding of the certificate-based authentication flow: generate key → CSR → k8s CSR object → approve → use cert.

COMMAND / YAML SNIPPET

```
# Step 1: Generate private key and CSR
openssl genrsa -out john.key 2048
openssl req -new -key john.key -out john.csr -subj '/CN=john/O=dev-team'

# Step 2: Create Kubernetes CSR object
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: john
spec:
  request: $(cat john.csr | base64 | tr -d '\n')
```

```
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
EOF

# Step 3: Approve the CSR
kubectl certificate approve john
kubectl get csr john

# Step 4: Extract approved certificate
kubectl get csr john -o jsonpath='{.status.certificate}' | base64 -d >
john.crt

# Step 5: Build kubeconfig for john
kubectl config set-credentials john --client-key=john.key --client-
certificate=john.crt --embed-certs
kubectl config set-context john-context --cluster=kubernetes --user=john
kubectl config use-context john-context
```

★ **EXAM TIP** CN= maps to username. O= maps to group. After creating kubeconfig, create a Role + RoleBinding to give john actual permissions.

Q20 Configure a static pod on the worker node that runs nginx. Then verify it appears as a pod in the cluster.

▸ **ANSWER**

Static pods are managed directly by the kubelet — not the API server. They are placed in /etc/kubernetes/manifests/ on the node. The kubelet watches this directory.

⌨ COMMAND / YAML SNIPPET

```
# Step 1: SSH to the target node
ssh worker-1

# Step 2: Find the staticPodPath
grep -i staticPodPath /var/lib/kubelet/config.yaml
# Default is /etc/kubernetes/manifests

# Step 3: Create static pod manifest
cat <<EOF > /etc/kubernetes/manifests/static-nginx.yaml
apiVersion: v1
kind: Pod
metadata:
  name: static-nginx
  labels:
    app: static-nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.21
    ports:
    - containerPort: 80
EOF
```

```
# Step 4: Verify from control plane (no kubectl apply needed)
kubectl get pods -A | grep static-nginx
# It appears as: static-nginx-worker-1 (node name appended)
```

⚠**WATCH OUT** Static pods show in 'kubectl get pods' but cannot be deleted via kubectl — delete the file from /etc/kubernetes/manifests/ to remove them.

CHAPTER 6 · TROUBLESHOOTING — 30%

The highest-weighted domain · Pods · Nodes · Services · Logs · Events · Resource constraints

★ EXAM CCFBF1

6.1 Universal Troubleshooting Flowchart

Symptom	First Command	Second Command	Common Fix
Pod Pending	kubectl describe pod <name>	kubectl get events	No nodes match selector / insufficient resources / PVC not bound
Pod CrashLoopBackOff	kubectl logs <pod> --previous	kubectl describe pod	Wrong command, bad config, missing secret/configmap
Pod Error/OOMKilled	kubectl describe pod	Check Events: OOMKilled	Increase memory limits
Service not reachable	kubectl get endpoints <svc>	kubectl describe svc	Selector mismatch / pod not running / wrong port
Node NotReady	kubectl describe node	ssh node; systemctl status kubelet	Kubelet down, disk pressure, network issue
ImagePullBackOff	kubectl describe pod	Check Events: Failed pull	Wrong image name/tag or private registry auth missing
CreateContainerError	kubectl describe pod	Check Events	Volume mount error, configmap missing, security context

Q21 A pod 'api-pod' is in OOMKilled state repeatedly. The application needs 512Mi memory. Fix the resource limits.

▸ **ANSWER**

OOMKilled means the kernel killed the container because it exceeded its memory limit. The fix is to increase the memory limit.

COMMAND / YAML SNIPPET

```
# Step 1: Confirm OOMKilled
kubectl describe pod api-pod | grep -A5 'Last State'
# Output: Reason: OOMKilled

# Step 2: Check current limits
kubectl describe pod api-pod | grep -A6 Limits

# Step 3: If pod is from a Deployment — edit the Deployment
kubectl edit deployment api-deployment
# Find resources section and update:
      resources:
        limits:
          memory: '512Mi'
          cpu: '500m'
        requests:
          memory: '256Mi'
          cpu: '200m'

# Step 4: Verify new pod starts without OOMKill
kubectl get pods -w
kubectl describe pod <new-pod-name> | grep -A6 Limits
```

PRO TIP Requests = minimum guaranteed resources (used for scheduling). Limits = maximum allowed (enforcement). OOMKilled always = limit too low.

Q22 A service 'db-svc' has no endpoints even though the database pods are running. Diagnose and fix.

▸ **ANSWER**

Empty endpoints means the service selector does not match any pod labels. This is the most common service issue in the CKA.

COMMAND / YAML SNIPPET

```
# Step 1: Check endpoints
kubectl get endpoints db-svc
# Output: NAME      ENDPOINTS   AGE
#         db-svc    <none>      5m   ← empty = problem

# Step 2: See what selector the service uses
kubectl describe svc db-svc | grep Selector
# Example: Selector: app=database

# Step 3: See what labels the pods actually have
kubectl get pods --show-labels | grep <pod-pattern>
# Example shows: app=db  ← MISMATCH! Service wants app=database

# Fix option A: Patch the service selector to match pods
```

```
kubectl patch svc db-svc -p '{"spec":{"selector":{"app":"db"}}}'

# Fix option B: Relabel the pods to match service
kubectl label pod <pod-name> app=database --overwrite

# Step 4: Verify endpoints populate
kubectl get endpoints db-svc
# Output should show: 10.244.x.x:5432
```

⚠**WATCH OUT** ALWAYS verify both: service selector AND pod labels. Also check the service port matches the container port. Both must be correct.

Q23 Check and fix a failing kubeapi-server. The cluster is completely unresponsive to kubectl commands.

▸ **ANSWER**

If kubectl commands fail entirely, the API server is down. It runs as a static pod — check its manifest for config errors.

⌨**COMMAND / YAML SNIPPET**

```
# Step 1: SSH to control plane node directly
ssh controlplane

# Step 2: Check if API server container is running
crictl ps | grep apiserver
crictl logs <apiserver-container-id>

# Step 3: Check static pod manifest for errors
cat /etc/kubernetes/manifests/kube-apiserver.yaml
# Common errors: wrong etcd endpoint, bad cert path, typo in flags

# Step 4: Check kubelet logs for manifest errors
journalctl -u kubelet -n 100 | grep -i error

# Step 5: If manifest has a typo — fix it
vi /etc/kubernetes/manifests/kube-apiserver.yaml
# Save and kubelet will automatically restart the pod

# Step 6: Watch for API server to recover
watch crictl ps | grep apiserver
# Then test:
kubectl get nodes
```

⚠**WATCH OUT** When kubectl is completely broken: use crictl (container runtime CLI) to inspect containers. It works without API server. Also check /var/log/pods/kube-system_kube-apiserver*/

Q24 Deploy a pod and use kubectl exec to run a debugging command. A pod is failing — use an ephemeral debug container to troubleshoot it.

▸ **ANSWER**

Ephemeral containers allow you to attach a debugging image to a running pod without modifying it. Essential for debugging distroless or minimal images.

COMMAND / YAML SNIPPET

```
# Regular exec (when container has shell)
kubectl exec -it <pod-name> -- /bin/bash
kubectl exec -it <pod-name> -c <container-name> -- sh

# Check resource usage
kubectl exec <pod-name> -- df -h
kubectl exec <pod-name> -- cat /etc/resolv.conf
kubectl exec <pod-name> -- env | grep -i password

# Ephemeral debug container (when main container has no shell)
kubectl debug -it <pod-name> \
  --image=busybox:1.28 \
  --target=<container-name>

# Debug a node by running a pod with host namespaces
kubectl debug node/<node-name> -it --image=ubuntu

# Copy a pod and add debug tools
kubectl debug <pod-name> -it \
  --image=ubuntu \
  --copy-to=debug-pod \
  --share-processes
```

★ **EXAM TIP** kubectl debug is your best friend for distroless containers. The busybox:1.28 image is reliable for nslookup, wget, and basic debugging.

Q25 Use kubectl top to identify which pod is consuming the most CPU across all namespaces. Metrics-server is not installed — install it first.

▸ **ANSWER**

kubectl top requires metrics-server. In the exam it may already be installed or you may need to install it.

COMMAND / YAML SNIPPET

```
# Check if metrics-server is running
kubectl get pods -n kube-system | grep metrics-server

# Install metrics-server if missing
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml

# If TLS verification fails in exam environment, patch it:
kubectl patch deployment metrics-server -n kube-system \
  --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-
","value":"--kubelet-insecure-tls"}]'

# Wait for metrics-server to be ready
```

```
kubectl rollout status deployment metrics-server -n kube-system

# Top commands
kubectl top pods -A --sort-by=cpu     # all namespaces, sorted by CPU
kubectl top pods -A --sort-by=memory  # sorted by memory
kubectl top nodes                     # node resource usage
kubectl top pod <name> --containers   # per-container breakdown
```

PRO TIP kubectl top pods shows current usage. kubectl describe node shows allocatable vs requested (scheduling). Both are used in exam scenarios.

CHAPTER 7 · ADVANCED TOPICS & MUST-KNOW SNIPPETS

Multi-container pods · Init containers · Probes · ConfigMaps · Secrets · ServiceAccount tokens

Q26 Create a pod with an init container that writes a file, and a main container that reads that file. Use an emptyDir volume to share data.

▸ **ANSWER**

Init containers run to completion BEFORE main containers start. They share volumes with main containers. Perfect for pre-loading config, waiting for services, DB migrations.

COMMAND / YAML SNIPPET

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
spec:
  volumes:
  - name: shared-data
    emptyDir: {}
  initContainers:
  - name: writer
    image: busybox
    command: ['sh', '-c', 'echo Hello from init > /data/message.txt']
    volumeMounts:
    - name: shared-data
      mountPath: /data
  containers:
  - name: reader
    image: nginx
    volumeMounts:
    - name: shared-data
      mountPath: /data
EOF

kubectl get pod init-demo
```

```
kubectl exec init-demo -- cat /data/message.txt
kubectl describe pod init-demo | grep -A5 'Init Containers'
```

★ **EXAM TIP** Multiple init containers run sequentially in order. If one fails, Kubernetes restarts it (per restartPolicy). Main containers only start when ALL inits Complete.

Q27 Add liveness and readiness probes to an nginx container. Liveness: HTTP GET /healthz port 80 after 15s. Readiness: TCP port 80 every 5s.

▸ **ANSWER**

Liveness probe = restart container if it fails. Readiness probe = remove from service endpoints if it fails (no traffic). Both are tested separately.

⌨ **COMMAND / YAML SNIPPET**

```
containers:
- name: nginx
  image: nginx
  ports:
  - containerPort: 80
  livenessProbe:
    httpGet:
      path: /healthz
      port: 80
    initialDelaySeconds: 15
    periodSeconds: 20
    failureThreshold: 3
  readinessProbe:
    tcpSocket:
      port: 80
    initialDelaySeconds: 5
    periodSeconds: 5
  startupProbe:   # optional: delays liveness until app starts
    httpGet:
      path: /healthz
      port: 80
    failureThreshold: 30
    periodSeconds: 10
```

□ **PRO TIP** Three probe types: httpGet (checks HTTP response code), tcpSocket (checks port open), exec (runs a command, checks exit code 0). Know all three.

Q28 Create a Secret from literal values and from a file. Then inject the Secret both as environment variables and as a volume mount.

▸ **ANSWER**

Secrets are base64 encoded (NOT encrypted by default). For environment injection use env.valueFrom.secretKeyRef. For file injection use volumes.

⌨ **COMMAND / YAML SNIPPET**

```
# Create secret from literals
kubectl create secret generic db-creds \
  --from-literal=username=admin \
```

```
  --from-literal=password=S3cur3P@ss

# Create secret from file
echo -n 'MyS3cretToken' > token.txt
kubectl create secret generic api-token --from-file=token=token.txt

# Use secret as ENV vars
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-creds
      key: username
- name: DB_PASS
  valueFrom:
    secretKeyRef:
      name: db-creds
      key: password

# Use ALL keys from secret as ENV vars
envFrom:
- secretRef:
    name: db-creds

# Use secret as volume (files in /secrets/)
volumes:
- name: secret-vol
  secret:
    secretName: db-creds
volumeMounts:
- name: secret-vol
  mountPath: /secrets
  readOnly: true
```

⚠ **WATCH OUT** Decode a secret: kubectl get secret db-creds -o jsonpath='{.data.password}' | base64 -d . Secrets in volumes auto-update when secret changes. Env vars do not.

Q29 Create a multi-container pod with a sidecar pattern: main container runs nginx, sidecar container runs a log-shipper that tails nginx logs.

▸ **ANSWER**

The sidecar pattern uses a shared volume (emptyDir or hostPath) between containers in the same pod. Both containers run simultaneously.

COMMAND / YAML SNIPPET

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: sidecar-pod
spec:
  volumes:
  - name: log-volume
```

```
    emptyDir: {}
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: log-volume
      mountPath: /var/log/nginx
  - name: log-shipper
    image: busybox
    command: ['sh', '-c', 'tail -f /logs/access.log']
    volumeMounts:
    - name: log-volume
      mountPath: /logs
EOF

# Exec into specific container in multi-container pod
kubectl exec -it sidecar-pod -c nginx -- bash
kubectl exec -it sidecar-pod -c log-shipper -- sh
kubectl logs sidecar-pod -c log-shipper
kubectl logs sidecar-pod --all-containers
```

★ **EXAM TIP** Always specify -c <container-name> when exec/logs in multi-container pods. Without -c it uses the first container. kubectl describe shows all containers.

CHAPTER 8 · CLUSTER COMPONENTS DEEP DIVE

API Server · Scheduler · Controller Manager · etcd · kubelet · kube-proxy · CoreDNS

8.1 Component Reference

Component	Runs On	Purpose	Config Location
kube-apiserver	Control plane	Central REST API hub — all components talk to it	Static pod: /etc/kubernetes/manifests/kube-apiserver.yaml
etcd	Control plane	Distributed key-value store — cluster state database	Static pod: /etc/kubernetes/manifests/etcd.yaml
kube-scheduler	Control plane	Assigns pods to nodes based on resources/affinity	Static pod: /etc/kubernetes/manifests/kube-scheduler.yaml
kube-controller-manager	Control plane	Runs controllers: deployment, node, replicaset, etc.	Static pod: /etc/kubernetes/manifests/kube-controller-manager.yaml
kubelet	Every node	Node agent — starts/stops containers per pod spec	Service: /var/lib/kubelet/config.yaml
kube-proxy	Every node	Network rules (iptables/ipvs) for Service routing	DaemonSet in kube-system

CoreDNS	Control plane	Cluster DNS — resolves service names to ClusterIPs	Deployment in kube-system
Container Runtime	Every node	Runs containers — containerd, CRI-O, Docker	systemd service

8.2 Diagnosing Component Failures

```
# Check all control plane pods
kubectl get pods -n kube-system

# Check component health (deprecated in newer k8s but still works in exam)
kubectl get componentstatuses

# Static pod logs (no API server needed)
crictl ps | grep scheduler
crictl logs <container-id>

# Kubelet logs
journalctl -u kubelet -n 100 --no-pager
systemctl status kubelet

# Check static pod manifests for misconfigs
ls /etc/kubernetes/manifests/
cat /etc/kubernetes/manifests/kube-scheduler.yaml

# kube-proxy logs
kubectl logs -n kube-system -l k8s-app=kube-proxy

# Check certificate expiry
kubeadm certs check-expiration
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -dates
```

CHAPTER 9 · CRITICAL YAML TEMPLATES

Copy-paste ready templates for the exam · Pod · Deployment · Service · Ingress · RBAC

9.1 Complete Pod Template with All Fields

```
apiVersion: v1
kind: Pod
metadata:
  name: full-pod
  namespace: default
  labels:
    app: myapp
    tier: frontend
spec:
  serviceAccountName: my-sa      # custom service account
```

```
  securityContext:              # pod-level security
    runAsUser: 1000
    fsGroup: 2000
  initContainers:
  - name: init-wait
    image: busybox
    command: ['sh', '-c', 'until nc -z db-svc 5432; do sleep 2; done']
  containers:
  - name: app
    image: myapp:v1
    ports:
    - containerPort: 8080
    env:
    - name: ENV_VAR
      value: 'hello'
    - name: SECRET_VAR
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: password
    envFrom:
    - configMapRef:
        name: my-configmap
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 256Mi
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 10
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
    volumeMounts:
    - name: data
      mountPath: /data
    securityContext:            # container-level security
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
  volumes:
  - name: data
    emptyDir: {}
  nodeSelector:
    disktype: ssd
  tolerations:
  - key: 'node-role'
    operator: Equal
    value: worker
    effect: NoSchedule
  restartPolicy: Always
```

9.2 Role and RoleBinding Templates

```
# Role (namespaced)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-manager
  namespace: dev
rules:
- apiGroups: ['']            # '' = core API group (pods, services, etc)
  resources: ['pods', 'pods/log', 'pods/exec']
  verbs: ['get', 'list', 'watch', 'create', 'delete']
- apiGroups: ['apps']        # for deployments, statefulsets, daemonsets
  resources: ['deployments']
  verbs: ['get', 'list', 'update', 'patch']

# RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-manager-binding
  namespace: dev
subjects:
- kind: User
  name: john
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: ci-bot
  namespace: devops
roleRef:
  kind: Role            # or ClusterRole
  name: pod-manager
  apiGroup: rbac.authorization.k8s.io
```

9.3 NetworkPolicy Templates

```
# Default DENY ALL ingress to a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: prod
spec:
  podSelector: {}   # matches all pods
  policyTypes:
  - Ingress

# Allow specific ingress from namespace + port
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-specific
  namespace: backend
```

```
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: frontend
      podSelector:   # AND condition (same block)
        matchLabels:
          role: api
    ports:
    - port: 5432
      protocol: TCP
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          name: monitoring
    ports:
    - port: 9090
```

CHAPTER 10 · TOP TIPS, TRICKS & COMMON GOTCHAS

What the exam really tests · Time-saving tricks · Mistakes that cost marks

10.1 Speed Tips — Save 20+ Minutes

Slow Way	Fast Way	Time Saved
Writing full deployment YAML	kubectl create deployment name --image=img --dry-run=client -o yaml	3-5 min
Manually typing kubectl config use-context	Copy-paste from question (always provided)	30 sec
Finding YAML structure from memory	kubectl explain pod.spec.containers.resources	2 min
Writing service YAML	kubectl expose deployment name --port=80 --type=NodePort	2-3 min
Creating RBAC resources	kubectl create role/clusterrole/rolebinding imperatively	3-4 min
Checking if fix worked	kubectl get pods -w (watch mode)	Ongoing
Navigating docs	Bookmark: kubectl cheatsheet on k8s.io	1-2 min

10.2 The 30 Most Important kubectl Commands

```
# ── GENERATION (exam gold) ──────────────────────────────────────
kubectl run pod-name --image=nginx $do                # pod yaml
kubectl create deploy name --image=nginx --replicas=3 $do  # deploy yaml
kubectl create service clusterip name --tcp=80:8080 $do
kubectl create configmap name --from-literal=k=v $do
kubectl create secret generic name --from-literal=k=v $do
kubectl create serviceaccount name $do
kubectl create role name --verb=get,list --resource=pods $do
kubectl create clusterrole name --verb=get --resource=pods $do
kubectl create rolebinding name --role=rolename --user=username $do

# ── EDITING ─────────────────────────────────────────────────────
kubectl edit deployment name             # opens in $KUBE_EDITOR
kubectl patch deployment name -p '{...}' # JSON patch
kubectl set image deployment/name container=newimage:tag
kubectl set env deployment/name KEY=value
kubectl label pod name key=value --overwrite
kubectl annotate pod name key=value

# ── INSPECTION ──────────────────────────────────────────────────
kubectl describe pod name | grep -A10 Events
kubectl logs name --previous --tail=50
kubectl exec -it name -- bash
kubectl port-forward svc/name 8080:80    # local tunnel
kubectl get pods -A -o wide              # all namespaces + IPs
kubectl get pods --field-selector=status.phase=Running
kubectl auth can-i list pods --as=user

# ── NAMESPACE TRICKS ────────────────────────────────────────────
kubectl get all -n kube-system           # everything in namespace
kubectl delete pod name $now             # force delete immediately
kubectl rollout restart deployment/name  # rolling restart
```

10.3 Common Exam Gotchas

⚠WATCH OUT FEE2E2

⚠WATCH OUT FEE2E2

⚠WATCH OUT FEF3C7

⚠WATCH OUT FEF3C7

★ EXAM CCFBF1

★ EXAM CCFBF1

□ PRO TIP FEF3C7

CHAPTER 11 · PREPARATION STRATEGY

Study plan · Practice environments · Resources · What to build · 4-week roadmap

11.1 4-Week Study Roadmap

Week	Focus	Daily Practice	Goal
Week 1	Core concepts + kubectl fluency	30 min kubectl drills — create/edit/delete every resource type	Typing kubectl commands without thinking
Week 2	Workloads + Networking + Storage	Build and break: deploy apps, create services, test DNS	Can solve W2 and W3 exam domains comfortably
Week 3	Cluster admin + RBAC + etcd	Full cluster setup with kubeadm in a VM, etcd backup/restore	Can manage a cluster end-to-end
Week 4	Troubleshooting + Mock exams	2 full timed mock exams per day, review wrong answers	Under exam time pressure, hitting 75%+ consistently

11.2 Practice Environments

Tool	Cost	Best For	Setup Time
killer.sh (CKA simulator)	~$30 (included with exam)	Closest to real exam environment — USE THIS	Minutes
Killercoda CKA scenarios	Free	Daily practice scenarios, guided labs	Minutes
kubeadm on VMs (VirtualBox/VMware)	Free	Real cluster setup, etcd practice, upgrades	2-4 hours
k3s / kind / minikube	Free	Fast local practice, not good for kubeadm/upgrade	Minutes
Play with Kubernetes (PWK)	Free	Quick browser-based multi-node cluster	Minutes
Labs.play-with-k8s.com	Free	Multi-node exploration without installation	Minutes

11.3 Official Allowed Resources During Exam

- https://kubernetes.io/docs — official documentation (allowed)
- https://kubernetes.io/blog — official blog (allowed)
- https://github.com/kubernetes — official GitHub (allowed)
- No Google, no Stack Overflow, no other tabs

★ EXAM CCFBF1

11.4 Day-of-Exam Checklist

1. Room ready: clean desk, no papers, webcam working, face visible, ID ready
2. One allowed browser tab open to kubernetes.io/docs before proctor connects
3. First 3 minutes: set alias k=kubectl, export do, enable autocomplete
4. Check context: kubectl config get-contexts — know your clusters
5. Read EVERY question fully before typing anything
6. Flag questions you are unsure about — return after completing others
7. Verify EVERY answer: kubectl get/describe to confirm it worked
8. Leave 15 minutes at end to revisit flagged questions

11.5 Topics to Practice Until Automatic

Topic	Times to Practice	Why
etcd backup and restore	10+ times	Always in exam, exact flags are hard to memorise
Cluster upgrade (kubeadm)	5+ times	Multi-step, easy to miss a step under pressure
RBAC (role/binding/SA)	10+ times	Always in exam, several variations possible
NetworkPolicy	10+ times	AND/OR logic trips up most candidates
Service creation and endpoint debug	10+ times	30% troubleshooting domain hits this constantly
kubectl exec / debug	Daily	Used in almost every troubleshooting scenario
Drain/uncordon workflow	5+ times	Node maintenance question is common
PV/PVC creation and binding	5+ times	Storage is 10% but YAML details catch people out

REMEMBER

The CKA is a hands-on performance exam. Reading is not enough. You must type commands until your fingers remember them. Every scenario in this guide should be practiced in a real cluster — not just read.

Pass rate improves dramatically after 4+ full mock exams on killer.sh. Good luck.